

Chat with PDF - Project Architecture Analysis Report

This document provides a detailed architectural analysis of the current "Chat with PDF" project structure. The project follows a Retrieval-Augmented Generation (RAG) architecture using FastAPI, Streamlit, PostgreSQL, pgvector, and Docker.

1. Project Overview

The application allows users to upload PDF documents and ask natural language questions regarding the uploaded content. The system extracts text from PDFs, chunks the content, converts it into vector embeddings, stores it in PostgreSQL using pgvector, and retrieves relevant chunks during question-answering.

2. Technology Stack

Component	Technology	Purpose
Frontend	Streamlit	User interaction interface
Backend	FastAPI	REST API framework
Database	PostgreSQL	Structured data storage
Vector Engine	pgvector	Semantic vector search
Containerization	Docker	Infrastructure management
Caching	Valkey	Performance optimization
Embeddings	Sentence Transformers	Text vectorization

3. Project Structure

- `streamlit_app.py` - Frontend UI for upload and chatbot interaction.
- `app/api/` - Contains upload and ask API endpoints.
- `app/services/` - Business logic such as embeddings and retrieval.
- `app/utils/` - Utility functions like chunking and PDF extraction.
- `app/models/` - Database model definitions.

app/schemas/ - Pydantic schemas for request validation.

app/core/ - Configuration and database setup.

4. End-to-End Workflow

- User uploads a PDF using Streamlit.
- FastAPI receives the uploaded document.
- PDF loader extracts textual content.
- Chunking utility splits content into semantic chunks.
- Embedding service generates vector embeddings.
- Embeddings are stored in PostgreSQL using pgvector.
- User submits a natural language question.
- Question embedding is generated.
- Similarity search retrieves relevant chunks.
- Retrieved context is passed to the LLM.
- LLM generates the final response.

5. Architectural Strengths

- Clear separation between API, service, utility, and data layers.
- Scalable backend folder structure.
- Efficient semantic search using pgvector.
- Dockerized infrastructure for consistency.
- Embedding and retrieval logic are modularized.
- FastAPI enables high-performance APIs.

6. Incomplete or Unclear Areas

llm.py - LLM integration appears incomplete.

schemas/ - Schema validation files appear mostly empty.

vector_store.py - ORM models may not yet be implemented.

config.py - Configuration centralization is incomplete.

cache integration - Valkey usage is not fully visible.

7. Industrial Recommendations

- Implement complete LLM integration using Ollama or OpenAI.
- Add Pydantic validation schemas for all APIs.
- Use environment variables for configuration management.
- Implement SQLAlchemy ORM models.
- Add metadata tracking such as page number and upload time.
- Introduce authentication for multi-user systems.
- Implement structured logging and monitoring.
- Add automated testing pipelines.
- Use Kubernetes in future for orchestration and scaling.

8. Final Assessment

The project demonstrates strong foundational understanding of Retrieval-Augmented Generation systems and modern AI backend architecture. The structure already resembles simplified industry-grade backend organization patterns. The overall architecture is modular, scalable, and suitable for future production-level expansion. Current uncertainties mainly relate to: LLM integration implementation Database schema details Cache utilization Multi-document isolation logic ORM model implementation